CS402 2025–2026: Assessed Coursework
Parallelizing a Computational Fluid Dynamics Solver

The CS402 module aims to provide both a foundation in High Performance Computing (HPC) with its role in science and engineering and practical hands-on experience developing research and industry-representative parallel applications. **The aim of this coursework is to parallelize a sequential computational fluid-dynamics (CFD) solver called Karman to run on multi-threaded CPU, GPU and cluster systems**.

The Karman Application

The Karman application is a computational fluid dynamics (CFD) solver that models two-dimensional incompressible fluid flow around an obstacle using numerical methods derived from the Navier–Stokes equations. The flow simulated by the application exhibits a von Kármán vortex street, a repeating pattern of alternating vortices formed in the wake of a blunt body due to unsteady flow separation. This phenomenon, named after the Hungarian mathematician Theodore von Kármán (1881–1963), arises naturally from the numerical solution of the governing equations and is characterised by periodic vortex shedding downstream of the obstacle.

Before discussing the specific tasks of this coursework, it is useful to first outline the governing equations, numerical methods, and solution scheme used in the code provided. A detailed background in computational fluid dynamics or physics is not assumed, and **you should not expect to understand all aspects of this**. This reflects a common scenario in high-performance computing, where HPC practitioners are often required to work with domain experts, perhaps developing a new code or using an existing scientific code. In such cases, gaining insight into the equations and numerical methods used, even at a high-level, is a valuable step toward understanding the program structure and identifying opportunities for parallelisation and optimisation.

0.1 Governing Equations

Karman computes: (1) Velocity components in the x and y directions and (2) Pressure field over a discretised 2D domain. The output is written to a binary file, which can be post processed to visualise flow features such as vortices, wakes, and pressure gradients. In the application, mass conservation is enforced through the incompressibility constraint where, u, v are velocity in the x and y dimensions respectively:

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0 \tag{1}$$

The momentum equations in the x- and y-directions are given by:

$$\frac{\partial u}{\partial t} + u\,\frac{\partial u}{\partial x} + v\,\frac{\partial u}{\partial y} = -\frac{\partial p}{\partial x} + \nu\nabla^2 u, \tag{2}$$

$$\frac{\partial v}{\partial t} + u\,\frac{\partial v}{\partial x} + v\,\frac{\partial v}{\partial y} = -\frac{\partial p}{\partial y} + \nu\nabla^2 v. \tag{3}$$

where p is pressure normalized by fluid density (i.e. $p \leftarrow p/\rho$ where ρ is density) , ν is the kinematic viscosity of the fluid, and t is time. x and y are the spatial coordinates. To solve the incompressible Navier–Stokes equations numerically, the Karman application employs a fractional-step (projection) method. In this approach, intermediate velocity fields are first computed by advancing the momentum equations without the pressure term. The tentative velocity components in the x- and y-directions are given by

$$f = u^n + \Delta t\left(-u\frac{\partial u}{\partial x} - v\frac{\partial u}{\partial y} + \nu\nabla^2 u\right) \tag{4}$$

$$g = v^n + \Delta t\left(-u\frac{\partial v}{\partial x} - v\frac{\partial v}{\partial y} + \nu\nabla^2 v\right) \tag{5}$$

where f and g denote the tentative velocity fields and Δt is the time-step size, u^n and v^n are the velocity components in the x- and y-directions, respectively, at time step n. The incompressibility constraint is enforced by solving a pressure Poisson equation derived from the continuity equation:

$$\nabla^2 p = \frac{1}{\Delta t}\left(\frac{\partial f}{\partial x} + \frac{\partial g}{\partial y}\right). \tag{6}$$

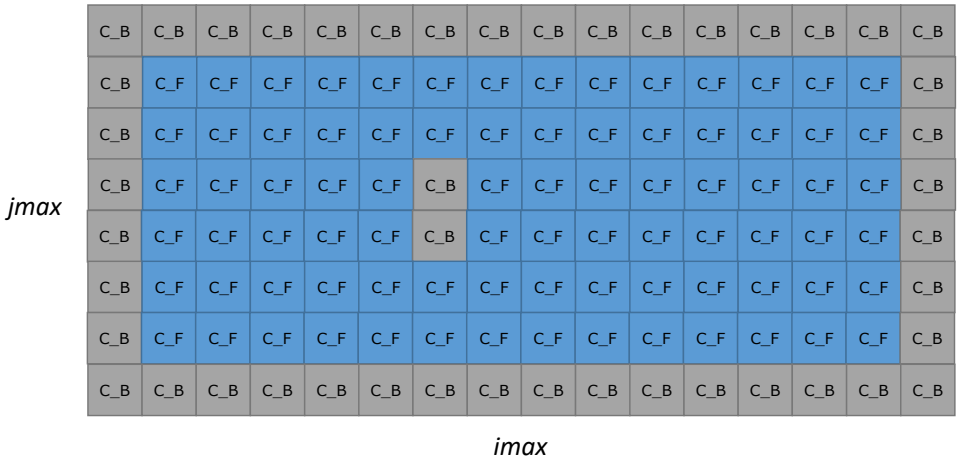


Figure 1: Karman Mesh cells

Once the pressure field has been obtained, the velocity components are corrected using the pressure gradients to yield a divergence-free velocity field:

$$u^{n+1} = f - \Delta t \frac{\partial p}{\partial x}, \tag{7}$$

$$v^{n+1} = g - \Delta t \frac{\partial p}{\partial y}. \tag{8}$$

Given that the solver models incompressible flow, no additional energy or temperature equations are required. Analytical solution for these equations are infeasible for realistic flows. Therefore, Karman uses numerical approximations to solve these equations over a discretised domain.

0.2 Spatial Discretization

The simulation domain is discretised using a uniform two-dimensional Cartesian grid of size $i_{max} \times j_{max}$. Each grid cell represents a control volume in which the flow variables are stored. The spatial step sizes in the x- and y-directions are denoted by Δx and Δy, respectively. Each cell (i, j) is classified as either a fluid cell (C_F) or an obstacle cell (C_B), allowing solid boundaries to be represented within the computational domain. The following quantities are stored at each grid cell: $u_{i,j}$ (x-velocity), $v_{i,j}$ (y-velocity), $p_{i,j}$ (pressure), $flag_{i,j}$ (cell type).

0.3 Stencil Computation

Spatial derivatives are approximated using second-order finite differences. For example, the discrete Laplacian of pressure at cell (i, j) is given by:

$$\nabla^2 p_{i,j} = \frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta x^2} + \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{\Delta y^2}. \tag{9}$$

Similarly, first-order derivatives are approximated using central differences:

$$\left.\frac{\partial u}{\partial x}\right|_{i,j} \approx \frac{u_{i+1,j} - u_{i-1,j}}{2\,\Delta x}, \quad \left.\frac{\partial v}{\partial x}\right|_{i,j} \approx \frac{v_{i+1,j} - v_{i-1,j}}{2\,\Delta x}, \quad \left.\frac{\partial u}{\partial y}\right|_{i,j} \approx \frac{u_{i,j+1} - u_{i,j-1}}{2\,\Delta y}, \quad \left.\frac{\partial v}{\partial y}\right|_{i,j} \approx \frac{v_{i,j+1} - v_{i,j-1}}{2\,\Delta y}. \tag{10}$$

This finite-difference computation is a *five-point stencil* computation where each new values at a cell depends on the cell itself and its four neighbors (north, south, east and west). Such as computation creates strong data locality, making the solver well suited for both CPU and GPU parallelization.

0.4 Numerical Method

The Karman application advances the solution in time using an explicit iterative time-stepping approach. Given velocity and pressure fields at time level n, updated values are computed at time level $n+1$. The process is repeated until a specified final simulation time is reached. At each time step, the governing equations are solved iteratively using neighbouring grid values, which leads to a high computational cost dominated by stencil-based operations.

As the Navier-Stokes equations are solved using a fractional-step method (see 4 and 5), the tentative velocity at each (i, j) cell, $f_{i,j}$ and $g_{i,j}$ are computed by advancing the momentum equations (2 and 3) **without the pressure term**. These intermediate velocities account for convection and diffusion but do not satisfy the incompressibility constraint. In practical terms, this means that at some grid cells more fluid is entering than leaving (or vice versa), which is not physically allowed for an incompressible fluid. To fix this a **pressure Poisson equation** is solved to compute a pressure field that corrects the tentative velocities so that the final velocity field satisfies the incompressibility constraint:

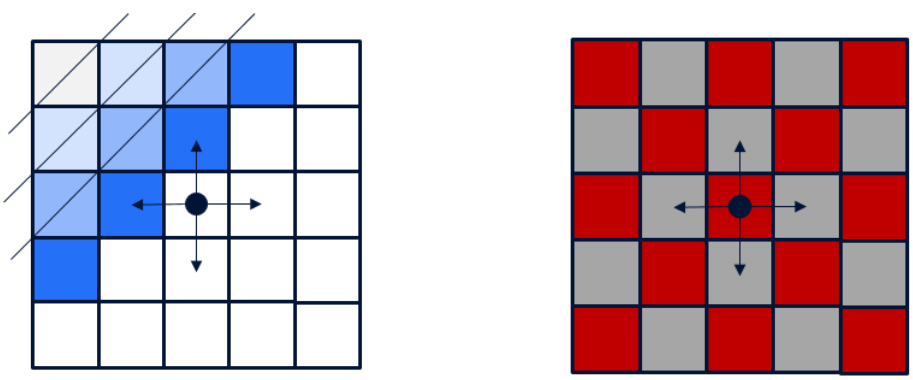
$$\nabla^2 p_{i,j} = \frac{1}{\Delta t}\left(\frac{f_{i,j} - f_{i-1,j}}{\Delta x} + \frac{g_{i,j} - g_{i,j-1}}{\Delta y}\right). \tag{11}$$

0.5 Successive Over-Relaxation (SOR)

The pressure Poisson equation is solved iteratively using the Successive Over-Relaxation (SOR) method. At iteration $k+1$, the pressure at an interior fluid cell (i, j) is updated as:

$$p_{i,j}^{k+1} = (1-\omega)\, p_{i,j}^{k} + \frac{\omega}{4}\left(p_{i+1,j}^{k} + p_{i-1,j}^{k+1} + p_{i,j+1}^{k} + p_{i,j-1}^{k+1} - h^2\, \mathrm{rhs}_{i,j}\right) \tag{12}$$

Here, ω is the relaxation parameter and h is the grid spacing. rhs represents the right-hand side of the pressure Poisson equation and corresponds to the the local violation of incompressibility and determines the pressure correction required at each grid cell. **This formulation introduces a sequential dependency that forms a wavefront pattern, limiting parallel efficiency. (see Figure 2a)**



(a) Wavefront dependency in standard SOR ordering.

(b) Red–Black ordering for SOR pressure updates.

Figure 2: Comparison of Red–Black SOR and standard SOR update patterns on a structured grid.

0.6 Red–Black SOR

To reduce data dependencies, **Karman uses a Red–Black SOR ordering** (see Figure 2b). The grid is partitioned into two interleaved sets of cells, denoted as red and black. Pressure values on red cells are updated using values from black cells, and vice versa:

$$p_{i,j}^{k+1} = (1-\omega)\, p_{i,j}^{k} + \frac{\omega}{4}\left(p_{i+1,j}^{*} + p_{i-1,j}^{*} + p_{i,j+1}^{*} + p_{i,j-1}^{*} - h^2\, \mathrm{rhs}_{i,j}\right) \tag{13}$$

where $(*)$ denotes values from the opposite colour set. This allows all red cells and all black cells to be updated independently, enabling efficient parallelisation. The Red-Black SOR is carried out as a inner iteration per outer time-step iteration.

0.7 Velocity Correction

Once the pressure field has converged, the velocity components are corrected:

$$u_{i,j}^{n+1} = f_{i,j} - \Delta t \, \frac{p_{i+1,j} - p_{i,j}}{\Delta x}, \quad v_{i,j}^{n+1} = g_{i,j} - \Delta t \, \frac{p_{i,j+1} - p_{i,j}}{\Delta y}. \tag{14}$$

These updates ensure that the corrected velocity field satisfies the incompressibility constraint.

0.8 Main Time-Stepping Loop

The Karman application advances the solution in time using a time-stepping approach. Rather than computing the flow field in a single operation, the solver updates the velocity and pressure fields incrementally using a sequence of small time steps. This is necessary because the governing equations describe unsteady flow and contain time-derivative terms. At each time step, the momentum equations are evaluated explicitly, followed by the iterative solution of a pressure Poisson equation to enforce incompressibility. By repeating this procedure, the solver captures the time-dependent evolution of the flow.

At each time step, the solver performs the following sequence. The initial serial code for Karman directly implements these steps in the given function names given in [...]:

1. **Compute a stable time-step size Δt**: [`set_timestep_interval()`]
   Calculate an appropriate time-step size. Ensures numerical stability of the explicit scheme.

2. **Compute tentative velocities f and g**: [`compute_tentative_velocity()`]
   Advances the flow forward in time without pressure, based on the current state. For each cell, compute a tentative new velocity based on the previous velocity and the stencil velocity values.

3. **Assemble the right-hand side of the pressure**: [`compute_rhs()`]
   Poisson equation measures how much the tentative velocities violate incompressibility. For each cell, calculate the RHS of the pressure equation.

4. **Solve the pressure Poisson equation**: [`poisson()`]
   Finds the pressure field needed to enforce mass conservation. For the entire pressure field, use Red/Black SOR to solve the Poisson equation. This takes a large number of (inner) iterations of the Red/Black process.

5. **Correct the velocity field**: [`update_velocity`]
   Applies the pressure correction to obtain a divergence-free velocity. For each cell, update the real velocity values based on the pressure matrix and the tentative velocity values.

6. **Apply boundary conditions**:[`apply_boundary_conditions`]
   Enforces physical constraints at walls and obstacles. This is done by updating the boundary velocities.

Only after all six steps, does the solver move to the next time step. This structure makes the application well suited for investigating performance optimisation and parallel programming techniques on modern high-performance computing systems.

Coursework 1 - OpenMP Parallelization

1.1 Getting Started

Download a copy of karman.zip file from the CS402 web page. Copy the zip file to `kudu-taught` and unzip it. Inside the downloaded folder there is a `Makefile`. To compile `karman` type at the command line:

```
module load cs402-mpi
make CC=mpicc
```

1.1.1 Run code and output

The executable `karman` will be generated in the same directory. Run the programe in the normal way, and `karman` will generate a default problem and solve it, writing the final solution out to the file `karman.bin`.

1.1.2 Runtime Options

The complete set of options that `karman` supports can be printed out using the following command:

```
./karman --help
karman. A simple computational fluid dynamics tutorial.
Usage: ./karman [OPTIONS]...
-h, --help Print a summary of the options
-V, --version Print the version number
-v, --verbose=LEVEL Set the verbosity level. 0 is silent
-x, --imax=IMAX Set the number of interior cells in the X direction
-y, --jmax=JMAX Set the number of interior cells in the Y direction
-t, --t-end=TEND Set the simulation end time
-d, --del-t=DELT Set the simulation timestep size
-i, --infile=FILE Read the initial simulation state from this file
(default is 'karman.bin')
-o, --outfile=FILE Write the final simulation state to this file (default is 'karman.bin')
```

1.1.3 View the Solution

You can take a look at the solution by using the `bin2ppm` program built as part of the `karman` build process. The program works using standard input and output streams, so you need to run as follows:

```
./bin2ppm < karman.bin > karman.ppm
```

The ppm file should be viewable in any standard Unix image viewer, but you can use the program `ppm2jpeg` to convert it to a JPEG file if you prefer. Figure3 shows such a snapshot. The dark regions represent low velocity while lighter regions indicate high velocity. Around the green circular obstacle there is adverse pressure near the surface of the obstacle, and the whitest areas appear at the topmost and bottommost points of the obstacle, where the flow accelerates the most. Downstream of the obstacle, a vortex forms as a swirling region of rotating fluid, creating a clearly visible wavy pattern after the obstacle. This downstream area is the wake region, characterized by low velocity and low pressure, which extends behind the obstacle and defines the unsteady flow behaviour.
You will notice that the output solution is the solution at the final time step of the main time-marching loop. Re-running `karmen` with an existing `karman.bin` file in the same directory will advance the solution starting from this state. Consequently you can use the script `batchrun.sh` to run `karman` to advance the solution, simulating for longer but with each run outputting the state of the flow fields to a `ppm` file. These files can be combined to generate a video of the flow using the following command:

```
ffmpeg -framerate 10 -i Run%d.ppm -c:v libx264 -pix_fmt yuv420p output.mp4
```

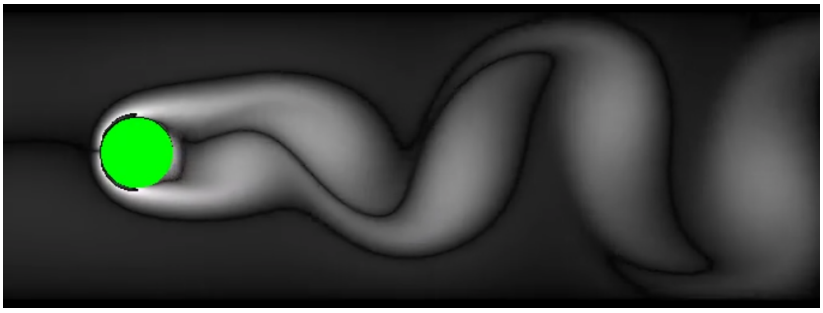


Figure 3: Karman output visualisation

1.1.4 Check the correctness of your parallel program

Once you start adding some code to `karman`, it's important that you check your results. You can do this with the `diffbin` program, that compares the final solution from the vanilla (serial) `karman` to your new parallel solution:

```
./diffbin karman.vanilla.bin karman.bin
```

`diffbin` by default checks both the **absolute error** and **relative error** between two solutions. **Note: Remember to rename the karman.bin file when you are doing multiple runs, otherwise it will be overwritten.** You may also find the extended utility `diffbin_ext` useful. It reports the maximum relative and absolute errors between solutions and indicates its location (grid point) and flow-field.

1.1.5 Change the problem size and Performance

The default mesh size set in `karman` is 660×120. This will produce a fairly low fidelity simulation, which you can notice from visualization generated by `bin2ppm`. It will approximately take 1 second to execute (on a single core of a 1.5GHz CPU). To create a bigger problem, use the -x and -y flags in `karman` to increase the number of cells in the X and Y dimensions. If you double the problem size the application will produce a slightly better fidelity solution, but the runtime will increase to about 4 seconds on the same system. Double again, will increase the time to about 8 seconds. You can try this your self using the `doublesize.sh` script.

These results demonstrate that the runtime increases significantly faster than the problem size, indicating poor scalability of the serial implementation. As higher resolution meshes are required to obtain accurate and physically meaningful solutions, single-core execution quickly becomes infeasible. Consequently, to enable larger problem sizes and reduce time-to-solution, it is necessary to exploit parallelism by distributing the computational workload across multiple processing cores.

1.1.6 Profile Code

You can profile the code and investigate where most of the time is spent in `karman`. To profile the code you will need to modify the `Makefile` so that `CFLAGS` has a `-pg` at the end of it. This will instruct the profiler `gprof` to record timing behaviour for the main routines in the code. Once you have changed the `Makefile` then you need to rebuild the code and run the program, where it will generate profiling information in a `gmon.out` file. This can be viewed with the following commands:

```
make clean all     #build
./karman           #run
gprof ./karman     #view profiling info with gprof
```

This generates a sorted list of the execution time spent in each function. It will give you a good idea as to where the majority of the runtime is spent. This is where you should focus your attention when thinking about parallelizing the code. We suggest you start your parallelisation of the program from the main computational part.

1.2 Paralleize with OpenMP for CPUs

For the first task of the coursework, **Utilizing *only* OpenMP, parallelise `karman` to run on multi-threaded CPUs.** Benchmark (1) **the runtime of the total program** and (2) **the runtime of each loop** over a varying number of threads. **Analyse the overhead of OpenMP.**

1.3 Performance Analysis Report

Write a report detailing the paralleization of the loop(s) in `karman` and the whole application. Include in the report: The `pragmas` used their influence on the performance of the code, iteration scheduling used, changes to the code done to parallelize. Present the analysis of benchmark times, with results presented as a graph or table and also analysis of OpenMP overhead. **The report should not exceed 4 pages of A4 with at least a 9pt font, excluding references.**

1.4 Deliverables

The coursework is released on the 12 January 2026. **The deadline for submitting the solution files and report via Tabula is Monday 10th February 2026 at 12 noon (Week 5).** The time you submit to Tabula will be taken as the official time of submission. Marking will be done anonymously, so please do not put your name in your program files or your report, instead include your University card number.

Submit the most optimal versions of the code you developed for Task 1.2 plus the Performance Analysis report. For the `karman` application, you can edit any file, but it should still compile and as run shown in Section 1.1. Only **OpenMP is permitted**. Your code will be ran using the same system and script as described in labs. Reasonable attempts will be made to get your code to run but you should check it first. Marking will be done over multiple problem sizes. **Final solution accuracy will be checked using `diffbin` and relative error compared to serial `karman` should be less than machine precision**.

Your **code** should be well documented and formatted, paying particular attention to readability and maintainability of the code. You can include any OpenMP statements or code that you tried and did not deliver performance. These should be commented out, and include a comment explaining why it did not work. For the **report** pay particular attention to the presentation, narrative and flow of argument, readability and structure.

Plagiarism

By now you will know the Department policy on allowable collaboration and reliance on outside sources. You may well find bits of code that are relevant to the coursework, if you do, make sure you acknowledge it and be aware that your submission will be marked taking this source into consideration. You may discuss the basic structure of your solution with colleagues, but the final code that is submitted must be your own.

Using generative AI tools, GAITs (e.g. Chat-GPT, Google Gemini, GitHub Copilot) is permitted under the condition that you acknowledge its use (in the report) specifying what was generated and where (in code comments and/or report). **Any prompts used to generate code/text and the resulting code/text must be submitted with the solution.** You must adhere to the new departmental guidelines introduced on the use of GAITs (see the updated student handbook). When assessing the coursework **you may be required to attend an in-person viva to demonstrate ownership of content you claim as your own**.

Be warned that **current generative AI frameworks and tools can both help and hinder your work**. They may produce considerable bugs in the source code which you will need to check and correct. **Particularly for parallel programming, AI generated code, if not checked and verified (especially with testing) is highly likely to lead to crashing, hanging and poor performance.** You will need a good understanding of parallel programming and your design/implementation to resolve these. The *technical debt* – the cost of taking shortcuts in software development, where expediency now leads to more work and higher costs later – can be significant. The need for systematic testing will increase and debugging time could be overwhelming.

Coursework 2 - Distributed Memory Parallelization

In this 2nd coursework, you will parallelise the `karman` solver using MPI to enable execution on distributed memory systems. You are required to parallelise the application to run efficiently across multiple compute nodes and, if you are attempting the final section, then also on GPU systems. The aim of this coursework is to expose students to realistic large-scale HPC programming challenges involving data distribution, communication, and heterogeneous architectures. Coursework 2 will be demanding and significantly more complex than Coursework 1, but it is also more open-ended and will allow you to demonstrate your parallel programming and optimization skills. It also carries a higher weight in marks awarded (**20% of the total module mark**).

2.1 Paralleize with MPI for CPU Cluster

Unlike the shared-memory OpenMP implementation developed in Coursework 1, this task requires explicit management of distributed memory using MPI. Each MPI process executes independently with its own address space, and all data exchanges between processes must be explicitly programmed. From your knowledge of `karman` you will already know the data structures and computational loops that operate over the global two-dimensional grid. These loops will need to be adapted to operate on sub-domains assigned to individual MPI ranks.

2.1.1 Domain Decomposition

A domain decomposition strategy must be employed to divide the computational grid among MPI ranks. A one-dimensional decomposition (e.g. splitting the domain in the y-direction) is recommended, though alternative strategies are permitted if correctly implemented and justified. Each MPI rank must compute updates only for its local sub-domain. Neighbouring ranks must exchange halo (ghost) cell data at each time step to ensure correctness of stencil-based computations.

You should use appropriate MPI point-to-point or collective communication routines and ensure that communication is correctly synchronised with computation. Blocking or non-blocking communication routines can be used. Any overlap of communication and computation should be clearly justified and must not compromise correctness.

2.1.2 File I/O and Checking the Solution

The issue of output generation needs to be addressed. Since each MPI rank holds only a portion of the domain, **a mechanism must be implemented to produce a single output file compatible with the existing post-processing tools**, essentially producing a single `karman.bin` file. Implementing full parallel I/O using MPI-IO is not required. Gathering data to a root process for output is sufficient, provided correctness and file compatibility are preserved. Correctness of the MPI-parallel implementation must again be verified using `diffbin`, with numerical differences remaining within machine precision.

2.2 Paralleize with MPI+OpenMP for CPU Cluster

Once you have a working and performant MPI implementation, **combine it with the paralleization techniques developed in Coursework 1 by introducing OpenMP `pragms` to loops**. This will parallelize the computation per MPI rank using multiple OpenMP threads. In principal on-node (or intra-node) paralleization with OpenMP will be orthogonal to the inter-node paralleization of MPI.

2.3 Performance

You should experiment with scaling the MPI parallel solution on an increasingly large number of processes and also by increasing the problem size. Additionally, useing the MPI+OpenMP paralleization, experiment with different combinations of MPI ranks and OpenMP threads and evaluate their impact on performance. Performance measurements should be obtained using appropriate timing routines (e.g. `MPI_Wtime` or `gettimeofday()` on root) rather than external timing tools. Does the hybrid implementation give improved time-to solution and scalability compared to pure MPI or pure OpenMP approaches on the target system ? Can you explain / justify the performance observed ?

2.4 Parallelize on GPUs

The final part of this coursework is designed to push your parallel programming skills towards accelerator programming, specifically parallleizing `karman` on NVIDIA GPUs. GPU programming can be demanding for a novice developer, particularly for a non-trivial code such as `karman`. However, if implemented well, you will observe that for the structured-mesh class of problems (such as in `karman`) the largest speed-ups could be achieved when runing on GPUs.

GPU parallelization is provided as an optional challenge task. You are not required to implement GPU parallelization for Karman, and no marks will be deducted if you choose not to do so. However, if you do implement it, you will receive a "bonus" of up to 10%, depending on the quality of your solution. For example, if you achieve 85% for your coursework based on OpenMP and MPI implementations and also develop a high-quality GPU parallelization, your final coursework mark will be 95% (85% + 10%). However, if you already score above 90% for OpenMP and MPI and also provide an excellent GPU solution, your final mark will be capped at 100%.

As GPU parallelization is an extra challenge task, limited guidance will be provided, and its development is largely up to your own initiative and effort.

2.5 Performance Analysis Report

Write a report detailing the design of the MPI, MPI+OpenMP (and CUDA) parallelizations of `karman`. Include in the report: The domain decomposition implemented, I/O strategy , and any optimizations implemented. Present the analysis of benchmark times, with results presented as a graph or table and also analysis of any overheads. Comment on the accuracy of the results at increasing scale. Explicitly present and discuss the parallelizations that led to the fastest time-to solution for a given problem size indicating raw run-times, Flops/s and/or Bandwidth utilized. You might find that presenting the work using roofline models will provide a comprehensive means of illustrating your performance results. **The report should not exceed 6 pages of A4 with at least a 9pt font, excluding references.**

2.6 Deliverables

The coursework is released on the 12 January 2026. **The deadline for submitting the solution files and report via Tabula is Monday 20th March 2026 at 12 noon (Week 11).** The time you submit to Tabula will be taken as the official time of submission. Marking will be done anonymously, so please do not put your name in your program files or your report, instead include your University card number.

Submit the most optimal versions of the code you developed for Task 2.1, 2.2 (and 2.4 if implemented) plus the Performance Analysis report. Provide clear documentation in the form of a README file on how to execute the application with any flags you have introduced. Again, you can edit any file in the `karman` application and only **MPI, OpenMP and/or CUDA is permitted**. **No other libraries permitted.** Your code will be run using the same system and script as described in labs. Reasonable attempts will be made to get your code to run but you should check it first. Marking will be done over multiple problem sizes. **Final solution accuracy will be checked using `diffbin` and relative error compared to serial `karman` should be less than machine precision**.

Your **code** should be well documented and formatted, paying particular attention to readability and maintainability of the code. For the **report** pay particular attention to the presentation, narrative and flow of argument, readability and structure.

Plagiarism

By now you will know the Department policy on allowable collaboration and reliance on outside sources. You may well find bits of code that are relevant to the coursework, if you do, make sure you acknowledge it and be aware that your submission will be marked taking this source into consideration. You may discuss the basic structure of your solution with colleagues, but the final code that is submitted must be your own.

Using generative AI tools, GAITs (e.g. Chat-GPT, Google Gemini, GitHub Copilot) is permitted under the condition that you acknowledge its use (in the report) specifying what was generated and where (in code comments and/or report). **Any prompts used to generate code/text and the resulting code/text must be submitted with the solution.** You must adhere to the new departmental guidelines introduced on the use of GAITs (see the updated student handbook). When assessing the coursework **you may be required to attend an in-person viva to demonstrate ownership of content you claim as your own**.

Be warned that **current generative AI frameworks and tools can both help and hinder your work**.

They may produce considerable bugs in the source code which you will need to check and correct. **Particularly for parallel programming, AI generated code, if not checked and verified (especially with testing) is highly likely to lead to crashing, hanging and poor performance.** You will need a good understanding of parallel programming and your design/implementation to resolve these. The *technical debt* – the cost of taking shortcuts in software development, where expediency now leads to more work and higher costs later – can be significant. The need for systematic testing will increase and debugging time could be overwhelming.